

Lab 4: Hardware Data Prefetcher Design and Analysis

Introduction

In this lab, I implement a GHB-based stride hardware data prefetcher. In a later step I improve the design by making it aware of system memory bandwidth constraints. Finally I create my own hybrid prefetcher, which chooses between a best-offset-prefetcher (Michaud 2016) and the previous strided GHB prefetcher.

Warm Up: Running the No-prefetching Baseline

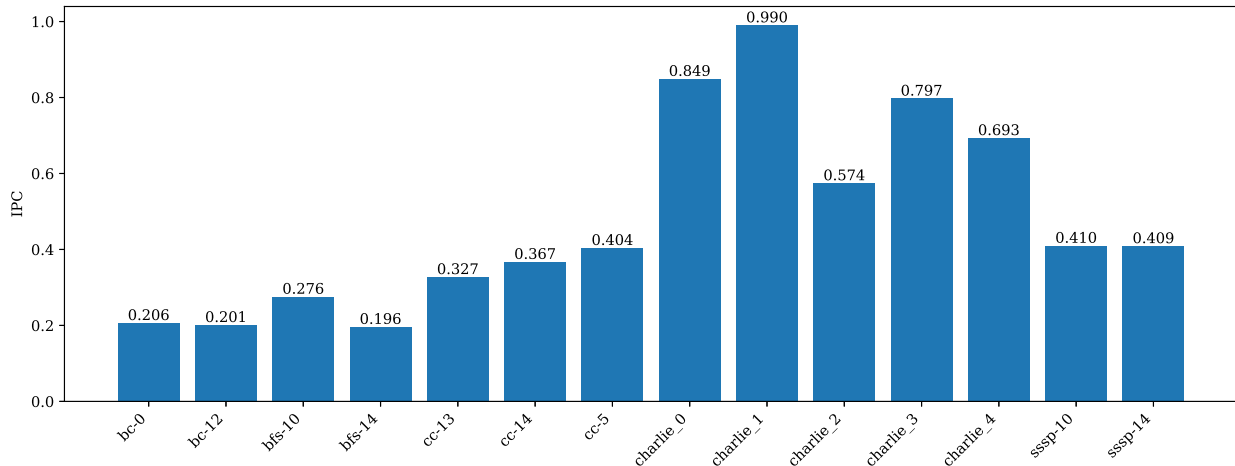


Figure 1: Baseline performance of the system without an L2C prefetcher on real-world CPU traces

The hardware prefetcher designs are evaluated on CPU traces collected from fourteen workloads sourced from graph analytics benchmarks and data-center workloads. Each workload was run for 50 million cycles following 10 million cycles of initial warmup using the Champsim microarchitectural simulator (Gober et al. 2022). The simulated system has a single out-of-order core and a single channel of DRAM running at a data rate of 4800 MT/s (“full BW”).

In fig. 1 is a plot of the instructions-per-cycle (IPC) for each workload. The `charlie_1` trace resulted in the highest IPC value, while the `bfs-14` trace got the lowest IPC value.

Task 1/4: Implementing GHB-based Stride Prefetcher

Implementation

In Task 1 I implemented a GHB-based stride prefetcher (Nesbit and Smith 2004) for the L2C as closely as possible to the original specification. I took special care to accurately replicate the bit-width of tags in the Index Table (IT) and pointers in the Global History Buffer (GHB). However in my own testing, I found that the IPC results for the exact replication are identical to those with a more relaxed replication of the paper with wider bit-widths (tbl. 1). First, this shows that the hardware budget proposed in the paper provides performance equal to an expensive prefetcher, but with a fraction of the resources. Second, this means that my replication of the original prefetcher isn’t sensitive to the specific sizings of the table entries.

Table 1: Per-trace IPC for GHB PC/CS with 20b and 56b wide IT tags (1 cpu, full BW)

trace	1	2	3	4	5	6	7	8	9	10	11	12	13	14
20b IPC	0.229	0.221	0.362	0.293	0.415	0.476	0.532	0.864	1.014	0.582	0.809	0.704	0.548	0.554
56b IPC	0.229	0.221	0.362	0.293	0.415	0.476	0.532	0.864	1.014	0.582	0.809	0.704	0.548	0.554

The per-workload IPC speedups, prefetching accuracies and prefetches per thousand instructions (PPKI) caused by the strided GHB prefetcher in the L2C are plotted in fig. 2. The IPC speedup is simply the ratio of the IPC measured with the prefetcher and the IPC without the prefetcher. Next, the accuracy is defined as the ratio of

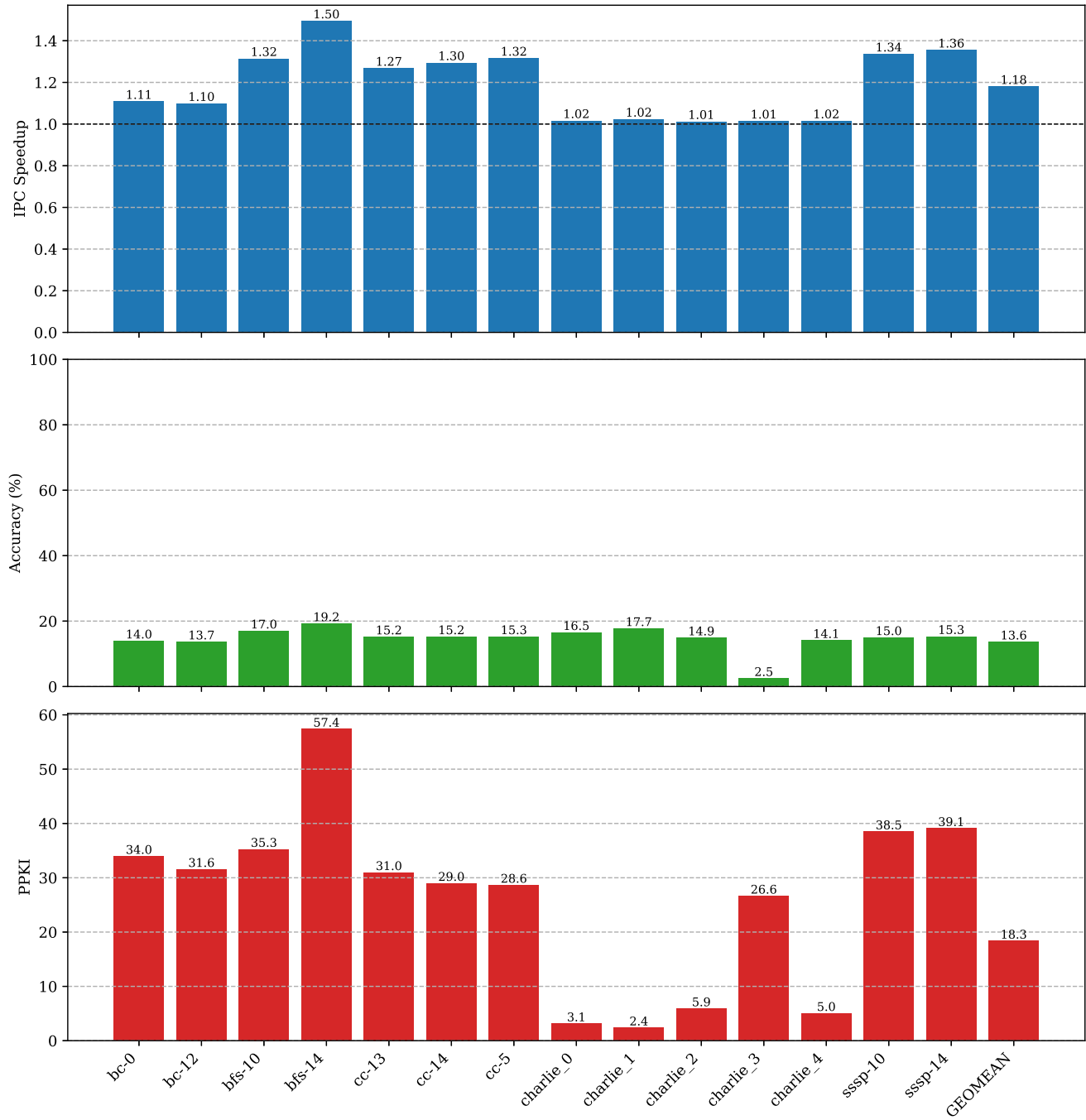


Figure 2: Task 1: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher relative to system without a prefetcher (1 CPU, full BW)

prefetches reported by Champsim logs as being “useful” (`l2c_prefetch_useful`) and the total number of issued prefetches (`l2c_prefetch_issued`).

Results

The utility of the prefetcher is highly heterogenous for the different types of workloads. As explained in the following paragraphs, this indicates that the strided GHB prefetcher is only useful for workloads with access patterns that play to its strengths.

GAP Workloads `bfs-14` was the workload with the largest IPC speedup. It was also the workload with the largest amount of prefetches issued, at the highest accuracy. The large number of prefetches issued indicate that the GAP workloads (graph analytics) can benefit a lot from a strided GHB prefetcher, considering they are most likely traversing neighboring nodes lists.

Charlie Workloads `charlie_2` got the lowest IPC speedup of all workloads. With the exception of `charlie_3`, very few prefetches were issued for this group of workloads, leading to very low IPC speedups overall. For `charlie_3`, the number of issued prefetches was much higher, but with very poor accuracy. This indicates that the charlie workloads (sourced from Google data centers) feature cache accesses that don’t suit a constant-stride prefetcher’s profile. Possibly these workloads resemble pointer-chasing or random accesses.

Task 2/4: Prefetching with Limited Main Memory Bandwidth

In this task, I investigate the effects of limited main memory bandwidth on the strided GHB prefetcher. The system with limited main memory bandwidth (“limited BW”) is identical to the system with full BW except that the main memory bandwidth is 800 MT/s, a sixth of the full BW.

Results

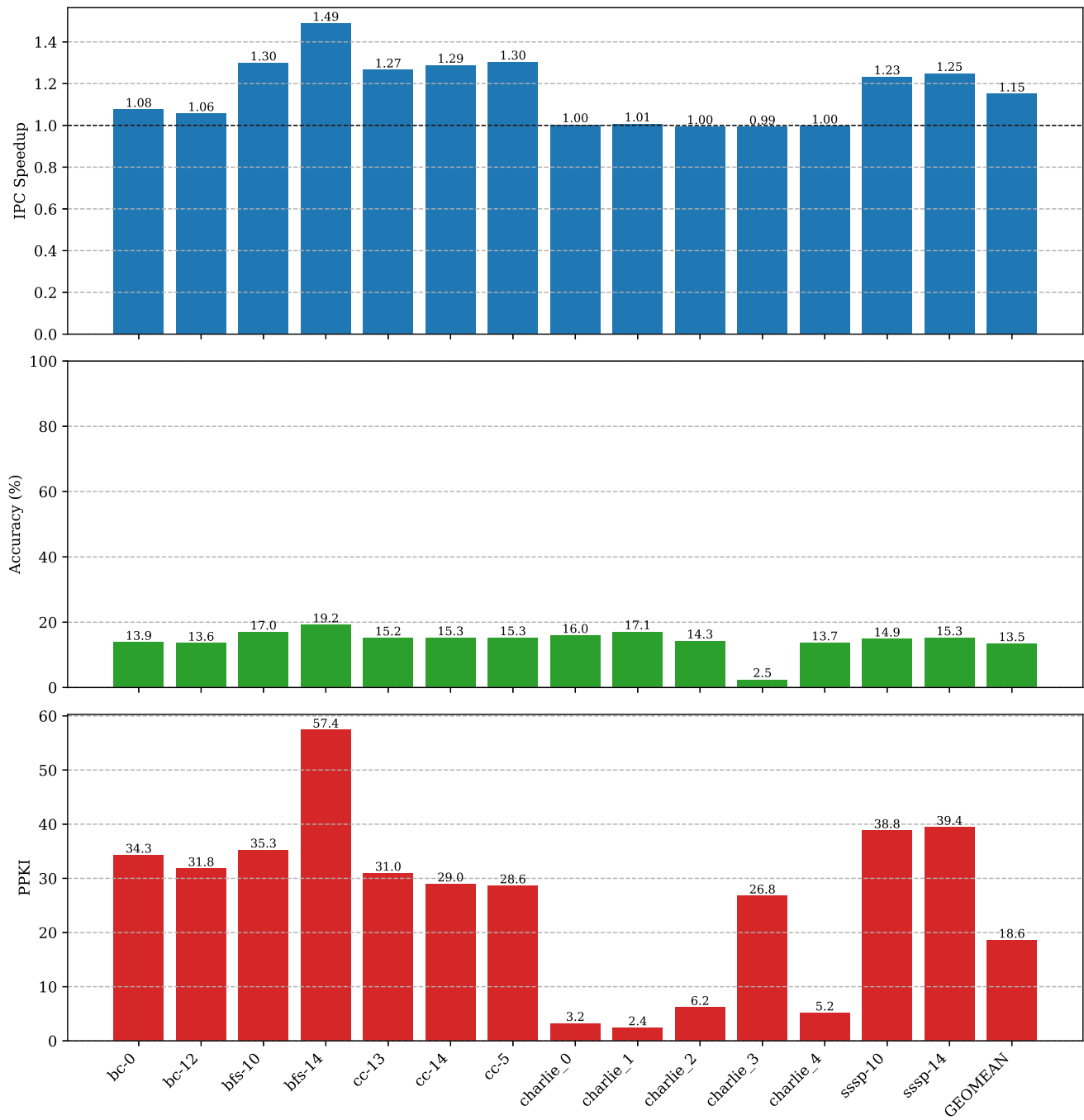


Figure 3: Task 2: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher relative to system without a prefetcher (1 CPU, limited BW)

Citations

- Gober, Nathan, Gino Chacon, Lei Wang, et al. 2022. *The Championship Simulator: Architectural Simulation for Education and Competition*. <https://arxiv.org/abs/2210.14324>.
- Michaud, Pierre. 2016. “Best-Offset Hardware Prefetching.” *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 469–80. <https://doi.org/10.1109/HPCA.2016.7446087>.
- Nesbit, K. J., and J. E. Smith. 2004. “Data Cache Prefetching Using a Global History Buffer.” *10th International Symposium on High Performance Computer Architecture (HPCA’04)*, 96–96. <https://doi.org/10.1109/HPCA.2004.10030>.